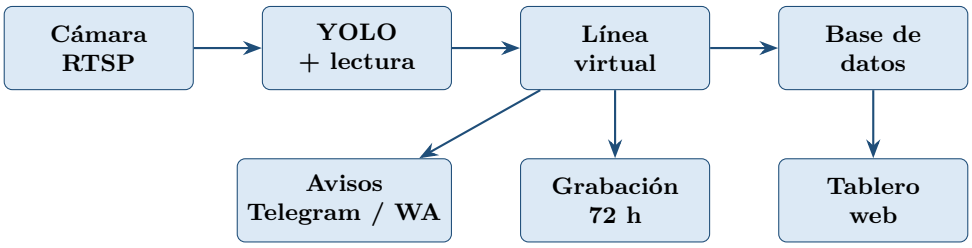
SARU

Sistema Automatizado de Reconocimiento de Unidades

Reporte de seguimiento — Prototipo 1

Metodología de prototipado evolutivo



Versión: 0.1
Fecha: 13 de noviembre de 2026
Organización: Grupo CISA
Elaboran: *(nombres por definir)*
Estado: Borrador para revisión del equipo
Repositorio: https://github.com/visioncisa/vision-cisa

Resumen

El control de entradas y salidas de las unidades del Metrobús en el Patio CCA depende del registro manual, lo que provoca movimientos perdidos, horas poco confiables y falta de evidencia cuando un registro se cuestiona. El proyecto SARU (*Sistema Automatizado de Reconocimiento de Unidades*), desarrollado por Grupo CISA, propone un sistema de visión artificial que detecta cada unidad en la puerta del patio, lee su número económico, decide si entra o sale mediante una línea virtual, registra el movimiento y avisa a los responsables. Todo el procesamiento ocurre en el borde, en una computadora NVIDIA Jetson Orin, con tres modelos YOLO en cascada optimizados con TensorRT.

El desarrollo sigue la metodología de **prototipado evolutivo**. Este documento reporta el **Prototipo 1**: la cadena completa de detección, lectura, cruce, registro, avisos, tablero web y grabación continua, que opera en vivo en el patio desde noviembre de 2026. Se presentan sus requerimientos, historias de usuario, arquitectura, protocolo de pruebas y el trabajo previsto para los siguientes prototipos.

Índice general

Resumen I

1. Introducción 1
- 1.1. Problemática . . . 1
- 1.2. Propuesta de solución . . . 2
- 1.3. Objetivos . . . 2
  - 1.3.1. Objetivo general . . . 2
  - 1.3.2. Objetivos específicos . . . 2
- 1.4. Justificación . . . 3

2. Marco teórico 4
- 2.1. Visión artificial y detección de objetos . . . 4
  - 2.1.1. YOLO . . . 4
  - 2.1.2. Modelos en cascada . . . 4
- 2.2. Seguimiento multiobjeto . . . 5
- 2.3. Lectura de caracteres por detección . . . 5
  - 2.3.1. Confirmación por votación . . . 5
- 2.4. Determinación de cruces con una línea virtual . . . 5
- 2.5. Computación en el borde . . . 6
  - 2.5.1. NVIDIA Jetson Orin y TensorRT . . . 6
- 2.6. Validación con modelos de lenguaje multimodales . . . 6
- 2.7. Tecnologías de soporte . . . 6

3. Estado del arte 7
- 3.1. Trabajos relacionados . . . 7
- 3.2. Comparación con SARU . . . 7

4. Metodología 9
- 4.1. Prototipado evolutivo . . . 9
- 4.2. Ciclo de cada prototipo . . . 9
- 4.3. Plan de prototipos . . . 9

5. Bosquejo general 11
- 5.1. Requerimientos funcionales . . . 11
- 5.2. Arquitectura general del prototipo . . . 13
  - 5.2.1. Arquitectura en capas . . . 13
- 5.3. Diagrama de base de datos . . . 14

6. Prototipos **15**
6.1. Prototipo 1: Sistema base en sitio . . . 15
6.1.1. Introducción . . . 15
6.1.2. Objetivo . . . 15
6.1.3. Especificaciones (historias de usuario) del Prototipo 1 . . . 15
6.1.4. Arquitectura del Prototipo 1 . . . 19
6.1.5. Componentes del Prototipo 1 . . . 19
6.1.6. Diseño de la interfaz del Prototipo 1 . . . 20
6.1.7. Desarrollo del Prototipo 1 . . . 20
6.1.8. Protocolo de pruebas del Prototipo 1 . . . 22
6.1.9. Conclusiones del Prototipo 1 . . . 25

7. Trabajo a futuro **26**
7.1. Prototipo 2: operación continua . . . 26
7.2. Prototipo 3: mejora de la lectura . . . 26
7.3. Pendientes del proyecto . . . 26

8. Conclusiones **28**

Referencias **29**

A. Evidencias de pruebas del Prototipo 1 **31**
A.1. LVC: línea virtual y cruce . . . 31
A.2. REG, AVS y REN: registro, avisos y rendimiento . . . 31
A.3. ADQ, DLN, WEB y GRB . . . 31

B. Glosario **32**
B.1. Proyecto . . . 32
B.2. Unidades . . . 32
B.3. Movimiento . . . 33
B.4. Lectura . . . 33
B.5. Avisos y evidencia . . . 34

Índice de figuras

4.1. Prototipado general del proyecto. 10

5.1. Arquitectura general del prototipo. 13
5.2. Modelo relacional de la base de datos. Las flechas van de la llave foránea a la tabla referida. 14

6.1. Arquitectura del Prototipo 1: flujo de una unidad desde la cámara hasta el registro. 19
6.2. Tablero principal: indicadores, gráficas y unidades dentro *(figura pendiente)* . 20
6.3. Video en vivo y lista paginada de cruces *(figura pendiente)* 21
6.4. Consulta y reproducción de grabaciones por hora *(figura pendiente)* 21
6.5. Aviso de cruce en Telegram con fotos y botones de validación *(figura pendiente)* 21
6.6. Diagrama de secuencia del registro de un cruce: detección, lectura, votación, cruce, espera, validación, registro y aviso *(figura pendiente)* 22

Índice de tablas

2.1. Tecnologías de soporte de SARU. 6

3.1. Comparación entre los trabajos relacionados y SARU. 7

5.1. Requerimientos funcionales. 11

6.1. Historia de usuario — Consultar unidades dentro. 16
6.2. Historia de usuario — Ver video en vivo. 16
6.3. Historia de usuario — Consultar cruces. 16
6.4. Historia de usuario — Recibir aviso de cruce. 16
6.5. Historia de usuario — Validar una lectura dudosa. 17
6.6. Historia de usuario — Registrar cada cruce una sola vez. 17
6.7. Historia de usuario — Registrar números fuera del padrón. 17
6.8. Historia de usuario — Consultar estancias y trayectoria. 17
6.9. Historia de usuario — Consultar grabaciones. 18
6.10. Historia de usuario — Revisar el estado del sistema. 18
6.11. Historia de usuario — Probar con video grabado. 18
6.12. Historia de usuario — Actualizar el padrón. 18
6.13. Historia de usuario — Reunir recortes para entrenamiento. 19
6.14. Componentes del Prototipo 1. 19
6.15. Cronología del desarrollo del Prototipo 1. 22
6.16. Nomenclatura de las pruebas del Prototipo 1. 23
6.17. Cobertura de pruebas del módulo LVC. 23
6.18. Cobertura de pruebas de los módulos REG, AVS y REN. 24
6.19. Cobertura de pruebas de los módulos ADQ, DLN, WEB y GRB. 24

Capítulo 1

Introducción

El Metrobús de la Ciudad de México opera con unidades que, al terminar o iniciar su servicio, entran y salen de patios de encierro. Saber qué unidades están dentro de un patio, desde cuándo y cuántas veces entraron o salieron en un día es información básica para la operación: permite planear la salida de unidades, detectar estancias anómalas y respaldar los registros ante una disputa entre las partes.

En el Patio CCA este control se hacía de forma manual. El proyecto SARU (*Sistema Automatizado de Reconocimiento de Unidades*), impulsado por Grupo CISA, busca automatizarlo con visión artificial: una cámara en la puerta del patio, una computadora de borde que procesa el video en tiempo real y un tablero web que muestra los movimientos al momento en que ocurren [2].

1.1. Problemática

El registro manual de entradas y salidas depende de que una persona anote cada unidad que cruza la puerta. Esa dependencia genera tres problemas:

- **Movimientos perdidos**, sobre todo en horas pico o cuando varias unidades cruzan al mismo tiempo.
- **Horas poco confiables**: la hora se anota tarde o se redondea, lo que altera el cálculo del tiempo que cada unidad permanece en el patio.
- **Falta de evidencia**: cuando un registro se cuestiona, no hay foto ni video que lo respalde.

Automatizar el registro tiene sus propias dificultades. El número económico está pintado en distintas zonas de la unidad (frente, costado o parte trasera) y la cámara no siempre lo ve completo; las unidades pueden detenerse sobre la puerta, maniobrar o cruzar varias a la vez; y la red del patio restringe la comunicación con servicios externos [5]. Un sistema útil debe resolver estos casos sin intervención humana constante y con un costo de operación bajo.

Por validar

Falta cuantificar el problema con datos del patio: cuántas unidades cruzan por día, cuántos movimientos se perdían con el registro manual y quién consume hoy esa información.

1.2. Propuesta de solución

Se propone un sistema de visión artificial que corre por completo en una computadora NVIDIA Jetson Orin instalada junto a la puerta del patio, sin depender de un servidor en la nube. A partir del video de la cámara, el sistema:

1. **Detecta** cada unidad con un modelo YOLO (M1) y la sigue cuadro a cuadro con ByteTrack.
2. **Lee** el número económico en dos pasos: un segundo modelo (M2) localiza la zona donde está pintado y un tercero (M3) lee los dígitos. Un número solo se confirma cuando varias lecturas coinciden.
3. **Decide** si la unidad entra o sale cuando su trayectoria atraviesa una línea virtual trazada sobre la imagen, y descarta las maniobras.
4. **Registra** cada cruce y cada estancia en una base de datos SQLite, y compara el número con el padrón oficial de unidades.
5. **Avisa** por Telegram y WhatsApp con hasta tres fotos del cruce; si la lectura es dudosa, la valida un modelo de lenguaje (Gemini) o una persona.
6. **Publica** un tablero web con video en vivo, cruces, unidades dentro y estancias, y **graba** 72 horas de video continuo como evidencia.

Los tres modelos se ejecutan en la GPU de la Jetson como motores TensorRT, y el sistema se despliega en tres contenedores Docker: visión, grabación y WhatsApp [6].

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar un sistema de visión artificial en el borde que registre de forma automática, con evidencia y sin captura manual, cada entrada y salida de unidades en la puerta del Patio CCA, y que ponga esa información a disposición de los responsables en el momento en que ocurre.

1.3.2. Objetivos específicos

- Detectar y seguir las unidades que aparecen en el video de la puerta, aunque haya varias a la vez.
- Leer el número económico de cada unidad en sus zonas delantera, lateral o trasera, y confirmarlo con varias lecturas.
- Determinar la dirección de cada cruce (entrada o salida) con una línea virtual, sin contar maniobras ni unidades detenidas.
- Registrar cruces y estancias en una base de datos, respetando el padrón oficial y conservando los números que no están en él.
- Notificar cada cruce con fotos y permitir su validación humana o automática.
- Ofrecer un tablero web para consultar los movimientos y conservar video continuo como evidencia.
- Validar el funcionamiento de cada prototipo con un protocolo de pruebas documentado.

1.4. Justificación

Un registro automático elimina la dependencia de una persona en la puerta, sella la hora con el reloj del sistema y guarda evidencia de cada movimiento: fotos del cruce y video continuo de las últimas 72 horas. Con ello, las estancias, los tiempos y los conteos diarios del patio pasan a ser datos confiables que pueden consultarse en cualquier momento.

La propuesta procesa todo **en el borde**. Esto reduce la dependencia de la red del patio, que ya bloquea algunos servicios externos, evita enviar video a terceros y mantiene un consumo bajo: entre 11 y 19 W según haya o no unidades en cuadro [6]. El uso de tres modelos en cascada (unidad, zona del número y dígitos) permite leer números pintados en distintas partes de la unidad, algo que un detector único de matrículas no resuelve.

En cuanto a viabilidad, el proyecto ya cuenta con el equipo instalado en el patio, modelos entrenados para cada etapa y un sistema que opera en vivo. El alcance de este documento comprende el primer prototipo funcional y la planeación de los siguientes. El cierre del proyecto está previsto antes del 30 de noviembre de 2026 [6].

Capítulo 2

Marco teórico

Este capítulo resume los conceptos en que se apoya SARU: la detección y el seguimiento de objetos en video, la lectura de caracteres, la determinación de cruces con una línea virtual, la computación en el borde y las tecnologías de soporte del sistema.

2.1. Visión artificial y detección de objetos

La detección de objetos consiste en localizar, dentro de una imagen, cada instancia de una clase de interés y delimitarla con un recuadro. Los detectores modernos se basan en redes neuronales convolucionales y se dividen en dos familias: los de dos etapas, que primero proponen regiones y después las clasifican, y los de una etapa, que predicen recuadros y clases en una sola pasada.

2.1.1. YOLO

YOLO (*You Only Look Once*) es un detector de una etapa que divide la imagen en una cuadrícula y predice, para cada celda, recuadros con su confianza y su clase en una sola evaluación de la red [13]. Al no requerir una etapa de propuesta de regiones, es lo bastante rápido para procesar video en tiempo real. SARU usa la implementación de Ultralytics [19] para sus tres modelos.

Cada detección tiene una **confianza** entre 0 y 1; el sistema descarta las que quedan por debajo de un umbral. Un umbral alto reduce los falsos positivos a cambio de perder detecciones dudosas; uno bajo hace lo contrario.

2.1.2. Modelos en cascada

En lugar de un solo modelo que lo resuelva todo, SARU encadena tres detectores especializados. Cada etapa trabaja sobre el recorte que entrega la anterior, de modo que los modelos posteriores procesan imágenes más pequeñas y con menos distractores:

M1 — unidad
: Detecta autobuses en el cuadro completo.

M2 — zona del número
: Dentro del recorte de cada unidad, localiza la zona donde está pintado el número y la clasifica como delantera, lateral o trasera.

M3 — dígitos
: Dentro de la zona, detecta cada dígito del 0 al 9 como una clase independiente.

2.2. Seguimiento multiobjeto

Detectar una unidad en un cuadro no basta: hay que saber que la unidad del cuadro siguiente es la misma. El enfoque de *seguimiento por detección* asocia las detecciones de cuadros consecutivos y asigna a cada objeto un identificador que se conserva mientras permanece en escena.

ByteTrack es un algoritmo de este tipo que asocia todas las detecciones, incluidas las de baja confianza, en lugar de descartarlas. Con ello recupera objetos parcialmente ocultos y reduce los cambios de identificador [21]. En SARU, el identificador de seguimiento permite acumular varias lecturas del número de una misma unidad y evaluar su trayectoria respecto de la línea virtual.

2.3. Lectura de caracteres por detección

Los sistemas de reconocimiento automático de matrículas suelen dividirse en detección del vehículo, localización de la placa y reconocimiento de caracteres [14]. Una alternativa para la última etapa es tratar cada carácter como un objeto: un detector localiza los dígitos y estos se ordenan por su posición horizontal para formar el número [8]. SARU aplica este enfoque con M3.

Antes de leer los dígitos, la imagen de la zona se mejora con CLAHE (*Contrast Limited Adaptive Histogram Equalization*), que ecualiza el contraste por regiones pequeñas y limita la amplificación del ruido [22]. Esto ayuda con números pintados en superficies con reflejos o sombras.

2.3.1. Confirmación por votación

Una sola lectura puede confundir dígitos parecidos (un 5 con un 6, un 8 con un 3) o perder uno. Por eso un número solo se **confirma** cuando varias lecturas de la misma unidad coinciden. Si un número leído es otro con dígitos de menos (por ejemplo, `504` dentro de `5014`), se prefiere el más largo.

2.4. Determinación de cruces con una línea virtual

Una **línea virtual** es un trazo definido por puntos en la imagen de la cámara. Para saber de qué lado de un segmento $\overline{PQ}$ se encuentra un punto B de la unidad, se usa el signo del producto cruz:

$$s = (Q_x - P_x)(B_y - P_y) - (Q_y - P_y)(B_x - P_x) \tag{2.1}$$

Un valor positivo indica un lado y uno negativo el otro. Un cambio de signo entre dos posiciones consecutivas indica un posible cruce. Para que el conteo sea robusto se agregan tres reglas:

- **Margen**: dentro de una franja alrededor de la línea el lado no cambia. Así, el temblor de la detección con la unidad detenida sobre la línea no produce cruces en cadena.
- **Cruce real**: con una línea finita, el cambio de lado solo cuenta si el segmento recorrido por la unidad corta algún tramo de la línea; rodearla por un extremo no es un cruce [6].
- **Maniobra y repetición**: si la unidad regresa antes de cierto tiempo, el movimiento se descarta; y un mismo número no se registra dos veces en un intervalo corto.

2.5. Computación en el borde

La computación en el borde procesa los datos cerca de donde se generan, en lugar de enviarlos a un centro de datos. Reduce la latencia, el consumo de ancho de banda y la exposición de datos sensibles [15]. Para video, evita transmitir de forma continua un flujo de alta resolución.

2.5.1. NVIDIA Jetson Orin y TensorRT

La familia Jetson Orin integra CPU y GPU en un módulo de bajo consumo pensado para inferencia en el borde [9]. TensorRT optimiza un modelo entrenado para una GPU concreta: fusiona capas, reduce la precisión numérica cuando es seguro hacerlo y genera un *motor* (`.engine`) [10]. Ese motor depende de la GPU y de la versión de TensorRT con que se creó, por lo que debe generarse en la propia Jetson.

2.6. Validación con modelos de lenguaje multimodales

Los modelos de lenguaje multimodales reciben imágenes y texto y responden en lenguaje natural. SARU usa Gemini [7] como segunda opinión: le envía la foto del cruce y el número leído, y el modelo indica si coinciden. Cuando hay duda, la decisión pasa a una persona por Telegram.

2.7. Tecnologías de soporte

Tabla 2.1: Tecnologías de soporte de SARU.

Tecnología	**Uso en el sistema**
OpenCV	Lectura del video RTSP y manejo de imágenes [11].
Flask	Servidor web del tablero y de la API de consulta [12].
SQLite	Base de datos local de cruces y estancias, en modo WAL para permitir lecturas mientras se escribe [17].
Docker Compose	Despliegue de los tres servicios del sistema en la Jetson.
Telegram Bot API	Avisos al grupo de operación y validación con botones [18].
Baileys	Cliente de WhatsApp Web para reenviar los avisos; no es la API oficial de WhatsApp [20].
FFmpeg	Grabación continua del video de la cámara sin recodificar.

Capítulo 3

Estado del arte

El problema de SARU se relaciona con el **reconocimiento automático de matrículas** (ALPR, por sus siglas en inglés), un campo con amplia literatura. La diferencia principal es que SARU no lee una placa con formato estándar, sino un número económico pintado en distintas zonas de la carrocería, y que además debe decidir la dirección del movimiento.

3.1. Trabajos relacionados

Revisión de métodos de ALPR
: Shashirangana et al. revisan los métodos de reconocimiento de matrículas y los organizan por etapa: detección de la placa, segmentación y reconocimiento de caracteres. Señalan como retos la variación de iluminación, el ángulo de la cámara y la oclusión parcial [14].

ALPR en tiempo real con YOLO
: Laroca et al. proponen un sistema de reconocimiento de matrículas basado por completo en detectores YOLO, con etapas en cascada para el vehículo, la placa y los caracteres, y muestran que puede operar en tiempo real [8].

3.2. Comparación con SARU

Tabla 3.1: Comparación entre los trabajos relacionados y SARU.

Característica	**ALPR general [14]**	**YOLO en cascada [8]**	**SARU**
Objeto que se lee	Matrícula	Matrícula	Número económico pintado
Zonas de lectura	Una (placa)	Una (placa)	Delantera, lateral o trasera
Dirección del movimiento	No aplica	No	Sí (línea virtual)
Procesamiento	Variable	Por verificar	En el borde (Jetson)
Evidencia y avisos	No aplica	No	Fotos, video y avisos

Por completar

Esta sección es preliminar. Falta agregar soluciones comerciales de control de acceso vehicular, trabajos sobre identificación de autobuses de transporte público y conteo de vehículos con líneas virtuales, y revisar que la descripción de cada fuente coincida con el artículo original.

Capítulo 4

Metodología

4.1. Prototipado evolutivo

El desarrollo de SARU sigue la metodología de **prototipado evolutivo**: el sistema se construye mediante versiones funcionales sucesivas, cada una de las cuales se entrega, se prueba y se usa como base de la siguiente [16]. A diferencia del prototipo desechable, cada versión no se descarta: se corrige y se amplía.

Esta metodología es adecuada para SARU por tres razones:

- **Los requerimientos cambian con el uso.** Muchos casos solo aparecen con video real del patio: unidades detenidas sobre la línea, varias unidades a la vez o números leídos con dígitos de menos. Cada prototipo en sitio revela casos nuevos.
- **Los componentes son heterogéneos.** Cámara, modelos de visión, lógica de cruce, base de datos, mensajería y web dependen unos de otros; validarlos juntos en una sola entrega es riesgoso.
- **El sistema ya opera.** Cada prototipo debe mantener el servicio funcionando mientras se mejora.

4.2. Ciclo de cada prototipo

Cada prototipo recorre las mismas etapas y se documenta con la misma estructura en el capítulo 6:

1. **Especificación**: requerimientos que atiende e historias de usuario con sus criterios de aceptación [1].
2. **Diseño**: arquitectura, componentes e interfaz.
3. **Desarrollo**: implementación y despliegue en la Jetson.
4. **Pruebas**: protocolo con casos identificados por módulo y evidencia de cada resultado.
5. **Retroalimentación**: los fallos y los casos nuevos se convierten en requerimientos del siguiente prototipo.

4.3. Plan de prototipos

Prototipo 1: sistema base en sitio
: La cadena completa de detección, lectura, cruce, registro, avisos, tablero web y grabación continua, operando en vivo en el patio. Es el que reporta este documento (sección 6.1).

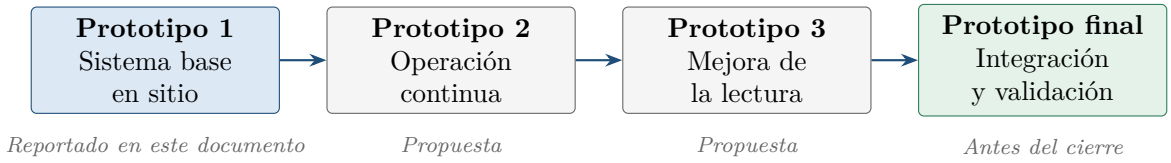


Figura 4.1: Prototipado general del proyecto.

Prototipo 2: operación continua

Llevar el sistema a operación permanente en el gabinete del patio con una Jetson de 16 GB [6]: arranque automático, reinicio ante fallas y verificación de que los avisos se entregan.

Prototipo 3: mejora de la lectura

Ampliar los datos de entrenamiento con los recortes del patio, reentrenar los modelos donde la lectura falla (por ejemplo, vistas no laterales) y medir la exactitud de cada etapa.

Prototipo final: integración y validación

Integrar los cambios de los prototipos anteriores y validar el sistema de extremo a extremo con pruebas en tiempo real, antes del cierre del proyecto.

Por validar

El contenido de los prototipos 2 y 3 es una propuesta basada en los pendientes registrados en los reportes del equipo. Debe confirmarse con la coordinación del proyecto.

Capítulo 5

Bosquejo general

5.1. Requerimientos funcionales

Los requerimientos se organizan por módulo. Cada requerimiento principal (`RF-NN`) agrupa requerimientos específicos (`RF-NN.M`). Los valores numéricos corresponden a la configuración actual del sistema [6].

Tabla 5.1: Requerimientos funcionales.

ID	Nombre	Descripción
RF-01	**Adquisición de video**	El prototipo leerá de forma continua el video de la cámara de la puerta.
RF-01.1	Conexión a la cámara	El prototipo leerá el video por RTSP con decodificación por hardware y, si esta falla, usará FFmpeg.
RF-01.2	Reconexión automática	Si se pierde la señal, el prototipo intentará reconectarse cada 2 segundos sin detener los demás servicios.
RF-01.3	Modo prueba	El prototipo podrá usar archivos de video en lugar de la cámara, con controles para avanzar, retroceder o saltar entre videos.
RF-01.4	Segunda cámara	El prototipo permitirá cambiar a una segunda cámara opcional desde el tablero.
RF-02	**Detección y seguimiento**	El prototipo detectará cada unidad presente en el cuadro y la seguirá mientras permanezca en escena.
RF-02.1	Detección de unidades	El prototipo detectará unidades con el modelo M1 y descartará las detecciones con confianza menor a 0.65.
RF-02.2	Seguimiento	El prototipo asignará a cada unidad un identificador de seguimiento y llevará por separado el número y el cruce de cada una, aunque haya varias a la vez.
RF-02.3	Límite de procesamiento	El prototipo limitará los cuadros por segundo de la detección a un valor configurable para no saturar la GPU.
RF-03	**Lectura del número económico**	El prototipo obtendrá el número económico de cada unidad detectada.
RF-03.1	Localización de la zona	El prototipo localizará con el modelo M2 la zona del número (delantera, lateral o trasera) y la asignará a la unidad que la contiene.
RF-03.2	Lectura de dígitos	El prototipo leerá los dígitos de la zona con el modelo M3, previa mejora de contraste con CLAHE.
RF-03.3	Confirmación	El prototipo confirmará un número con 2 lecturas iguales si está en el padrón, o con 4 si no lo está.

Tabla 5.1: Requerimientos funcionales (continuación).

ID	Nombre	Descripción
RF-03.4	Número más largo	Si un número leído está contenido en otro más largo, el prototipo conservará el más largo.
RF-03.5	Topes de lectura	El prototipo hará como máximo 2 lecturas por unidad y 4 por cuadro, y releerá cada 2 segundos una unidad cuyo número ya esté confirmado.
RF-03.6	Recortes	El prototipo guardará recortes de las unidades para revisión y para ampliar los datos de entrenamiento.
RF-04	**Determinación de cruces**	El prototipo decidirá si una unidad entra o sale del patio.
RF-04.1	Línea virtual	El prototipo comparará la posición de la base del recuadro de la unidad con una línea virtual configurable.
RF-04.2	Margen	El prototipo no cambiará el lado de una unidad que se encuentre a menos de 30 px de la línea.
RF-04.3	Maniobras	El prototipo descartará el movimiento de una unidad que regrese antes de 20 segundos.
RF-04.4	Repetición	El prototipo no registrará el mismo número dos veces en menos de 60 segundos.
RF-04.5	Registro único	Tras un cruce, el prototipo esperará a que la unidad termine de pasar (de 2 a 25 segundos) y registrará un solo evento con el mejor número y las mejores fotos.
RF-05	**Registro de movimientos**	El prototipo guardará cada cruce y cada estancia en la base de datos.
RF-05.1	Evento de paso	El prototipo registrará la unidad, la dirección, la puerta, la zona leída, la confianza, la foto y la hora de cada cruce.
RF-05.2	Estancias	El prototipo abrirá una estancia con cada entrada, la cerrará con la salida correspondiente y calculará su duración.
RF-05.3	Padrón	El prototipo sincronizará las unidades válidas desde el archivo del padrón en cada arranque.
RF-05.4	Fuera del padrón	El prototipo registrará tal cual los números confirmados que no estén en el padrón, marcados como tales.
RF-05.5	Día operativo	El prototipo contará como un mismo día los movimientos entre las 3:00 a.m. y las 2:59 a.m. del día siguiente.
RF-06	**Avisos y validación**	El prototipo notificará cada cruce a los responsables.
RF-06.1	Aviso por Telegram	El prototipo enviará al grupo de operación un aviso con la dirección, el tipo, la empresa y hasta 3 fotos separadas al menos 1 segundo.
RF-06.2	Validación humana	El aviso incluirá botones para que un validador confirme o corrija la lectura.
RF-06.3	Validación automática	El prototipo podrá enviar la foto a Gemini para revisar la lectura antes de registrarla.
RF-06.4	Aviso por WhatsApp	El prototipo reenviará los mismos avisos a un número de WhatsApp configurado.
RF-06.5	Modo sin avisos	El prototipo permitirá desactivar los avisos para pruebas.
RF-07	**Tablero web**	El prototipo ofrecerá un tablero web de consulta.
RF-07.1	Indicadores	El tablero mostrará indicadores, gráficas, unidades dentro y estancias.
RF-07.2	Video en vivo	El tablero mostrará el video de la cámara a 10 cuadros por segundo y 960 px de ancho.
RF-07.3	Lista de cruces	El tablero listará los cruces con búsqueda por número o empresa, filtro por fechas y páginas de 10.

Tabla 5.1: Requerimientos funcionales (continuación).

ID	Nombre	Descripción
RF-07.4	Trayectoria	El tablero mostrará el recorrido de una unidad.
RF-07.5	Estado del sistema	El prototipo informará su estado (conexión, cuadros por segundo, contadores) en una ruta de salud.
RF-08	**Grabación continua**	El prototipo conservará el video de la cámara como evidencia.
RF-08.1	Grabación por hora	El prototipo grabará el video sin recodificar en archivos de una hora y conservará las últimas 72 horas.
RF-08.2	Consulta de grabaciones	El tablero permitirá consultar y reproducir las grabaciones por hora.

5.2. Arquitectura general del prototipo

5.2.1. Arquitectura en capas

SARU se organiza en capas, cada una con una responsabilidad. Esta separación permite probar cada parte por separado, cambiar un servicio externo sin tocar la lógica de visión y que varias personas trabajen en módulos distintos sin pisarse.

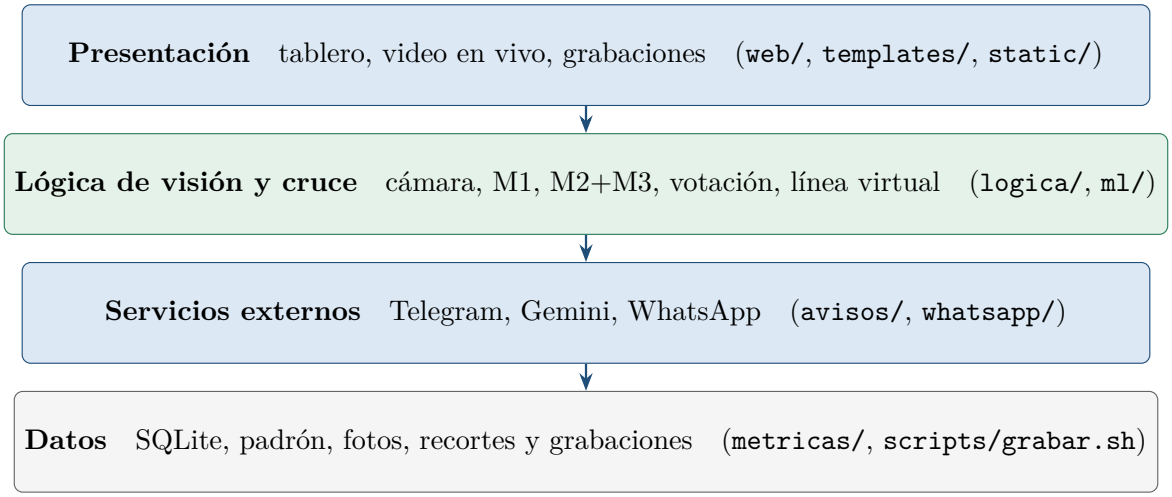


Figura 5.1: Arquitectura general del prototipo.

1. **Presentación**: Flask sirve el tablero, el video en vivo, las fotos, las grabaciones y una API JSON. Las rutas solo leen datos; no contienen lógica de visión ni de almacenamiento.
2. **Lógica de visión y cruce**: la cámara publica el cuadro más reciente; un hilo ejecuta M1 y el seguimiento, y otro ejecuta M2 y M3. La línea virtual decide los cruces y elige el mejor número.
3. **Servicios externos**: envío de avisos a Telegram y WhatsApp y validación con Gemini. Todos se pueden desactivar sin afectar el registro.
4. **Datos**: un solo módulo accede a la base SQLite; el padrón se lee de un archivo CSV; fotos, recortes y grabaciones se guardan en el disco de la Jetson.

El sistema se despliega con Docker Compose en tres servicios: `cisa-vision` (todas las capas anteriores), `cisa-grabacion` (grabación continua) y `cisa-whatsapp` (puente de

WhatsApp). La grabación es independiente de la detección: una falla en una no detiene a la otra [6].

5.3. Diagrama de base de datos

El modelo relacional se organiza en dos ejes: los **catálogos** (tipos de unidad, puertas y unidades) y los **movimientos** (eventos de paso y estancias). La duración y el estatus de cada estancia no se escriben: la base los calcula a partir de las horas de entrada y salida.

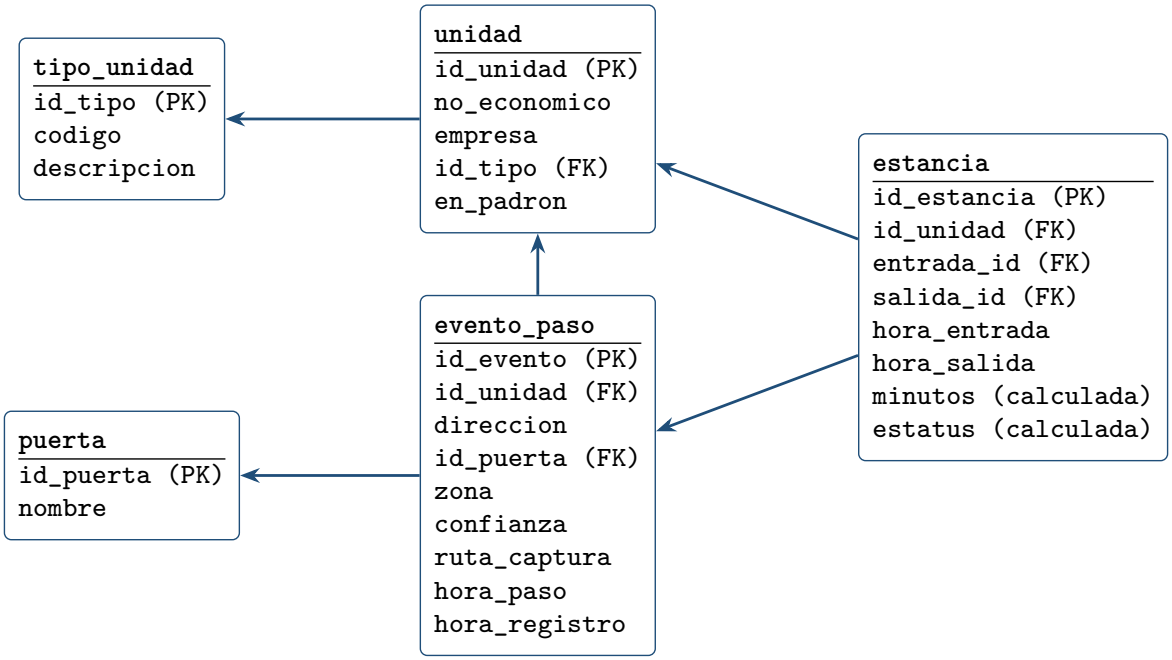


Figura 5.2: Modelo relacional de la base de datos. Las flechas van de la llave foránea a la tabla referida.

Sobre estas tablas se definen vistas para las consultas del tablero: unidades dentro en este momento, historial y promedio de estancias por empresa, y números detectados fuera del padrón.

Capítulo 6

Prototipos

6.1. Prototipo 1: Sistema base en sitio

6.1.1. Introducción

El Prototipo 1 constituye la base funcional de SARU: implementa la cadena completa desde la cámara hasta el registro y el aviso, sobre la que se construirán los prototipos siguientes. Resulta de la fusión, el 3 de noviembre de 2026, de dos versiones previas del sistema (la del repositorio y la que corría en la Jetson), seguida de correcciones con video real y de su puesta en operación en el patio [4, 5]. Establece la arquitectura en capas, el esquema de la base de datos, los tres modelos en cascada y la lógica de cruce que se reutilizarán en las siguientes iteraciones.

6.1.2. Objetivo

Implementar un prototipo funcional que, a partir del video de la cámara de la puerta del Patio CCA, detecte cada unidad, lea su número económico, determine si entra o sale, registre el movimiento con evidencia y lo notifique, y que ofrezca un tablero web para consultar los movimientos y las grabaciones.

6.1.3. Especificaciones (historias de usuario) del Prototipo 1

Las historias se escriben desde el punto de vista de cuatro tipos de usuario: **operador del patio**, **validador**, **coordinación** y **personal técnico**.

Por validar

Las historias se derivaron del sistema existente y de los reportes del equipo. Deben confirmarse con los usuarios reales del patio.

ID	Historia de usuario — Consultar unidades dentro
HU-01	Como operador del patio, quiero ver qué unidades están dentro y desde cuándo, para saber de qué unidades dispongo.
Prioridad	**Criterios de aceptación**
Alta	▪ El tablero muestra las unidades con estancia abierta y los minutos que llevan dentro. ▪ La lista se actualiza sin recargar la página.

Tabla 6.1: Historia de usuario — Consultar unidades dentro.

ID	Historia de usuario — Ver video en vivo
HU-02	Como operador del patio, quiero ver el video de la puerta en vivo, para comprobar lo que la cámara está viendo.
Prioridad	**Criterios de aceptación**
Alta	▪ El video se muestra a 10 cuadros por segundo y 960 px de ancho. ▪ Si se pierde la cámara, el tablero indica “Sin señal”.

Tabla 6.2: Historia de usuario — Ver video en vivo.

ID	Historia de usuario — Consultar cruces
HU-03	Como operador del patio, quiero consultar los cruces registrados y buscarlos por número, empresa o fecha, para revisar un movimiento concreto.
Prioridad	**Criterios de aceptación**
Alta	▪ La lista muestra número, empresa, dirección, zona, hora y foto. ▪ Permite buscar por número o empresa y filtrar por fechas. ▪ Muestra 10 cruces por página, con navegación entre páginas.

Tabla 6.3: Historia de usuario — Consultar cruces.

ID	Historia de usuario — Recibir aviso de cruce
HU-04	Como operador del patio, quiero recibir un aviso con fotos cada vez que una unidad entra o sale, para enterarme sin estar frente al tablero.
Prioridad	**Criterios de aceptación**
Alta	▪ El aviso indica número, dirección, tipo y empresa. ▪ Incluye hasta 3 fotos de momentos distintos, separadas al menos 1 segundo. ▪ Se envía por Telegram y, si está configurado, por WhatsApp.

Tabla 6.4: Historia de usuario — Recibir aviso de cruce.

ID	Historia de usuario — Validar una lectura dudosa
HU-05	Como validador, quiero confirmar o corregir el número de un cruce dudoso desde el aviso, para que el registro quede correcto.
Prioridad	**Criterios de aceptación**
Media	▪ El aviso incluye botones para confirmar o corregir. ▪ Solo los usuarios autorizados pueden validar. ▪ Antes del aviso, Gemini puede revisar la foto si está activado.

Tabla 6.5: Historia de usuario — Validar una lectura dudosa.

ID	Historia de usuario — Registrar cada cruce una sola vez
HU-06	Como coordinación, quiero que cada paso de una unidad genere un solo registro con el número correcto, para que los conteos sean confiables.
Prioridad	**Criterios de aceptación**
Alta	▪ Un cruce se registra cuando la unidad termina de pasar, con el número más leído y las mejores fotos. ▪ Una unidad detenida sobre la línea no genera registros. ▪ Una maniobra (regreso antes de 20 s) no genera registro. ▪ El mismo número no se registra dos veces en menos de 60 s. ▪ Con varias unidades a la vez, cada una lleva su propio cruce.

Tabla 6.6: Historia de usuario — Registrar cada cruce una sola vez.

ID	Historia de usuario — Registrar números fuera del padrón
HU-07	Como coordinación, quiero que un número leído de forma consistente se registre aunque no esté en el padrón, para no perder movimientos ni inventar unidades.
Prioridad	**Criterios de aceptación**
Media	▪ Con 4 lecturas iguales, el número se registra tal cual, marcado como fuera del padrón. ▪ No se corrige a un número parecido del padrón. ▪ El tablero permite consultar los números fuera del padrón.

Tabla 6.7: Historia de usuario — Registrar números fuera del padrón.

ID	Historia de usuario — Consultar estancias y trayectoria
HU-08	Como coordinación, quiero consultar las estancias y el recorrido de una unidad, para conocer cuánto tiempo permanece en el patio.
Prioridad	**Criterios de aceptación**
Media	▪ Cada estancia muestra hora de entrada, hora de salida y duración en minutos. ▪ El día operativo empieza a las 3:00 a.m. ▪ Se muestra el promedio, mínimo y máximo de estancia por empresa.

Tabla 6.8: Historia de usuario — Consultar estancias y trayectoria.

ID	Historia de usuario — Consultar grabaciones
HU-09	Como coordinación, quiero reproducir el video de una hora concreta, para revisar un cruce cuestionado.
Prioridad	**Criterios de aceptación**
Alta	▪ Hay un archivo por hora de las últimas 72 horas. ▪ El tablero lista las grabaciones por cámara y permite reproducirlas. ▪ La grabación continúa aunque falle la detección.

Tabla 6.9: Historia de usuario — Consultar grabaciones.

ID	Historia de usuario — Revisar el estado del sistema
HU-10	Como personal técnico, quiero conocer el estado del sistema, para detectar fallas sin entrar a la Jetson.
Prioridad	**Criterios de aceptación**
Alta	▪ Una ruta de salud informa conexión de la cámara, cuadros por segundo y contadores. ▪ Si la cámara se desconecta, el sistema intenta reconectarse cada 2 segundos.

Tabla 6.10: Historia de usuario — Revisar el estado del sistema.

ID	Historia de usuario — Probar con video grabado
HU-11	Como personal técnico, quiero ejecutar el sistema con un video en lugar de la cámara, para probar cambios sin afectar la operación.
Prioridad	**Criterios de aceptación**
Media	▪ Con un archivo de video como fuente, el sistema funciona igual que con la cámara. ▪ El tablero permite avanzar, retroceder y saltar entre videos. ▪ Los avisos pueden desactivarse durante las pruebas.

Tabla 6.11: Historia de usuario — Probar con video grabado.

ID	Historia de usuario — Actualizar el padrón
HU-12	Como personal técnico, quiero dar de alta o de baja unidades editando un archivo, para mantener el padrón al día sin tocar la base de datos.
Prioridad	**Criterios de aceptación**
Media	▪ El padrón se lee de un archivo CSV en cada arranque. ▪ Si una fila es inválida, el sistema no arranca e indica la línea con el error.

Tabla 6.12: Historia de usuario — Actualizar el padrón.

ID	Historia de usuario — Reunir recortes para entrenamiento
HU-13	Como personal técnico, quiero que el sistema guarde recortes de las unidades, para ampliar los datos con que se entrenan los modelos.
Prioridad	**Criterios de aceptación**
Media	▪ Se guarda un recorte por unidad cada cierto número de segundos. ▪ El tablero muestra estadísticas de los recortes guardados.

Tabla 6.13: Historia de usuario — Reunir recortes para entrenamiento.

6.1.4. Arquitectura del Prototipo 1

El Prototipo 1 aplica la arquitectura en capas del capítulo 5. Un solo proceso de Python (`cisa-vision`) arranca la cámara, lanza dos hilos de visión y el servidor web. La cámara publica el cuadro más reciente, del que leen la detección, el video en vivo y las capturas. El primer hilo ejecuta M1 y el seguimiento; el segundo, M2 y M3 sobre los recortes. Cuando un número confirmado cruza la línea, el sistema espera a que la unidad termine de pasar, valida, registra y avisa. La grabación lee la cámara por separado.

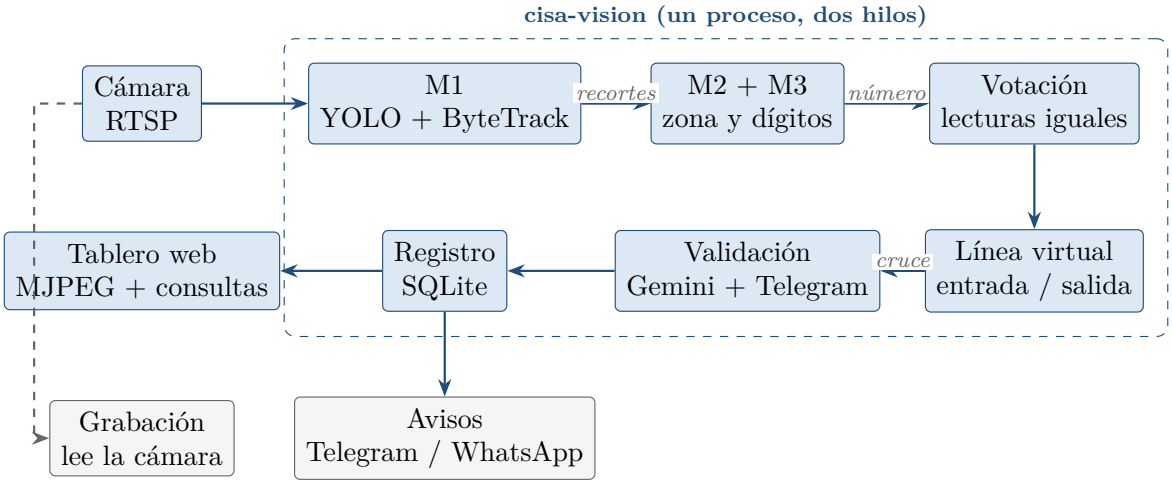


Figura 6.1: Arquitectura del Prototipo 1: flujo de una unidad desde la cámara hasta el registro.

6.1.5. Componentes del Prototipo 1

Tabla 6.14: Componentes del Prototipo 1.

Componente	Capa	Responsabilidad
`config.py`	Configuración	Único lector de la configuración; valida variables, modelos, carpetas y padrón al arrancar.
`pipe_stream_v2.py`	Arranque	Crea la cámara, lanza los hilos y el servidor web.
`logica/camara.py`	Visión	Lectura RTSP, respaldo con FFmpeg, reconexión y modo prueba.
`logica/estado.py`	Visión	Cuadro más reciente y estado compartido del sistema.

Tabla 6.14: Componentes del Prototipo 1 (continuación).

Componente	Capa	Responsabilidad
`ml/modelos.py`	Visión	Carga y calentamiento de los motores TensorRT.
`ml/hilos.py`	Visión	Hilo de detección (M1) e hilo de lectura (M2 y M3); registro único por cruce.
`ml/reconocimiento.py`	Visión	Recortes, lectura de dígitos, validación del número y guardado de recortes.
`ml/linea_virtual.py`	Visión	Lado de la línea, margen, maniobras y evaluación del cruce.
`metricas/schema.sql`	Datos	Tablas, índices y vistas.
`metricas/storage.py`	Datos	Único acceso a la base; sincronización del padrón, altas y consultas.
`avisos/telegram.py`	Servicios externos	Avisos y validación por Telegram.
`avisos/gemini.py`	Servicios externos	Revisión automática de la foto.
`avisos/whatsapp.py`, `whatsapp/`	Servicios externos	Puente hacia WhatsApp Web (Node.js y Baileys).
`web/rutas.py`	Presentación	Rutas del tablero, API, video y archivos.
`templates/`, `static/`	Presentación	Páginas, estilos y JavaScript del tablero.
`scripts/grabar.sh`	Datos	Grabación continua y rotación de archivos.

6.1.6. Diseño de la interfaz del Prototipo 1

La interfaz del Prototipo 1 tiene tres páginas: el tablero principal, el video en vivo con la lista de cruces y la consulta de grabaciones. Además, los avisos llegan a Telegram y WhatsApp.

Figura pendiente

Tablero principal: indicadores, gráficas y unidades dentro

Guarda la imagen en `figuras/06_prototipos/p1-tablero.png` y sustituye este recuadro por `\figura{06_prototipos/p1-tablero.png}{...}{fig:p1-tablero}`

Figura 6.2: Tablero principal: indicadores, gráficas y unidades dentro *(figura pendiente)*

6.1.7. Desarrollo del Prototipo 1

El Prototipo 1 se consolidó entre el 3 y el 12 de noviembre de 2026. La tabla 6.15 resume los cambios principales según los reportes del equipo.

Figura pendiente

Video en vivo y lista paginada de cruces

Guarda la imagen en `figuras/06_prototipos/p1-livevideo.png` y sustituye este recuadro por `\figura{06_prototipos/p1-livevideo.png}{...}{fig:p1-livevideo}`

Figura 6.3: Video en vivo y lista paginada de cruces *(figura pendiente)*

Figura pendiente

Consulta y reproducción de grabaciones por hora

Guarda la imagen en `figuras/06_prototipos/p1-grabaciones.png` y sustituye este recuadro por `\figura{06_prototipos/p1-grabaciones.png}{...}{fig:p1-grabaciones}`

Figura 6.4: Consulta y reproducción de grabaciones por hora *(figura pendiente)*

Figura pendiente

Aviso de cruce en Telegram con fotos y botones de validación

Guarda la imagen en `figuras/06_prototipos/p1-aviso.png` y sustituye este recuadro por `\figura{06_prototipos/p1-aviso.png}{...}{fig:p1-aviso}`

Figura 6.5: Aviso de cruce en Telegram con fotos y botones de validación *(figura pendiente)*

Tabla 6.15: Cronología del desarrollo del Prototipo 1.

Fecha	Cambio
3 nov	Fusión de las dos versiones previas: varias unidades a la vez y topes de procesamiento (repositorio) con margen de línea, espera de 25 s, avisos por WhatsApp y padrón de 396 unidades (Jetson). La versión fusionada pasa 16 de 16 pruebas del simulador, y la detección sube de 13.4 a 22.1 cuadros por segundo en modo de 40 W [4].
4 nov	Correcciones con video real: tres fallas que impedían registrar unidades bien leídas; los números fuera del padrón ahora se registran y se muestran en el tablero.
5–9 nov	Sistema en vivo en el patio: tres días sin fallas. Los avisos de Telegram no salen porque la red del patio bloquea el servicio; se preparó la solicitud para habilitarlo [5].
9–10 nov	Panel de cruces en la página de video en vivo, con búsqueda, filtros y paginación de 10.
10 nov	Incidente: avisos de WhatsApp que llegan sin poder leerse en el teléfono destinatario [3].
11 nov	Hasta 3 fotos por aviso; visor de fotos; videos de prueba del más reciente al más antiguo.
12 nov	Línea virtual de varios puntos que termina en sus extremos y compara la base del recuadro de la unidad. Probada en simulación; pendiente con video real. Solo en el paquete `SARU_V4` [6].

Versiones del código

Existen al menos tres versiones del Prototipo 1: la del repositorio, el paquete `SARU_V4` (con la línea de varios puntos) y la copia de la Jetson. Este documento describe la del repositorio salvo que se indique lo contrario. Definir una sola fuente de verdad es parte del trabajo a futuro (capítulo 7).

Figura pendiente

Diagrama de secuencia del registro de un cruce: detección, lectura, votación, cruce, espera, validación, registro y aviso

Guarda la imagen en `figuras/06_prototipos/p1-secuencia-cruce.png` y sustituye este recuadro por `\figura{06_prototipos/p1-secuencia-cruce.png}{...}{fig:p1-secuencia}`

Figura 6.6: Diagrama de secuencia del registro de un cruce: detección, lectura, votación, cruce, espera, validación, registro y aviso *(figura pendiente)*

6.1.8. Protocolo de pruebas del Prototipo 1

Cada caso de prueba se identifica con una sigla de módulo y un número consecutivo; por ejemplo, `LVC-01` es el primer caso de las pruebas de línea virtual y cruce. La columna **Aprobado** indica **Sí**, **No** o **Pendiente** (sin evidencia todavía). La evidencia de cada caso se reúne en el apéndice A.

Tabla 6.16: Nomenclatura de las pruebas del Prototipo 1.

Sigla	Módulo	Qué verifica
ADQ	Adquisición de video	Conexión, reconexión y modo prueba.
DLN	Detección y lectura	Detección de unidades, lectura y confirmación del número.
LVC	Línea virtual y cruce	Dirección del cruce, margen, maniobras, varias unidades y registro único.
REG	Registro	Eventos, estancias, padrón y números fuera del padrón.
AVS	Avisos y validación	Entrega de avisos y validación.
WEB	Tablero web	Páginas, consultas y video en vivo.
GRB	Grabación	Archivos por hora, rotación y consulta.
REN	Rendimiento	Cuadros por segundo, consumo y estabilidad.

Módulo de línea virtual y cruce (LVC)

Objetivo: verificar que el sistema registra una sola entrada o salida por cada paso real, y ninguna cuando la unidad se detiene, maniobra o rodea la línea.

Componentes que se prueban: `ml/linea_virtual.py` y el registro único de `ml/hilos.py`.

Entorno: simulador del equipo que reproduce cruces con reloj y modelos falsos [4, 6]. No incluye los modelos YOLO ni video real. El simulador no está en el repositorio.

Tabla 6.17: Cobertura de pruebas del módulo LVC.

ID	Caso	Resultado esperado	Aprobado	Observaciones
LVC-01	Una unidad entra.	Una entrada.	Sí	Simulador.
LVC-02	Una unidad sale.	Una salida.	Sí	Simulador.
LVC-03	Paso rápido (2 s) y lento (40 s).	Un registro en cada caso.	Sí	Simulador.
LVC-04	La unidad regresa a los 5 s.	Maniobra, sin registro.	Sí	Simulador.
LVC-05	La unidad regresa a los 15 s.	Maniobra, sin registro.	Sí	Fallaba en las versiones previas.
LVC-06	Unidad detenida sobre la línea con temblor.	Sin registro.	Sí	20 de 20 corridas.
LVC-07	Unidad detenida con vaivén lento.	Sin registro.	Sí	20 de 20 corridas.
LVC-08	Dos unidades lado a lado.	Un registro por unidad.	Sí	Fallaba en la versión de la Jetson.
LVC-09	Una unidad estacionada y otra entra.	Solo la entrada de la segunda.	Sí	Simulador.
LVC-10	Tres unidades a la vista, solo una cruza.	Un solo registro.	Sí	Simulador.
LVC-11	Una entra y otra sale a la vez.	Una entrada y una salida.	Sí	Simulador.
LVC-12	La unidad rodea la línea por un extremo.	Sin registro.	Sí	Solo línea de varios puntos (`SARU_V4`).
LVC-13	Entrada y salida con la línea de varios puntos y video real.	Un registro por paso.	Pendiente	Validar en la Jetson.

Módulos de registro, avisos y rendimiento (REG, AVS, REN)

Objetivo: verificar que los cruces quedan registrados, que los avisos llegan y que el sistema opera de forma estable en la Jetson.

Tabla 6.18: Cobertura de pruebas de los módulos REG, AVS y REN.

ID	Caso	Resultado esperado	Aprobado	Observaciones
REG-01	Unidad fuera del padrón leída de forma consistente.	Se registra con su número y aparece en el tablero.	Sí	Video real, 4 nov.
REG-02	Memoria tras 300 unidades.	Ningún estado sin uso.	Sí	Simulador: 0 registros sin uso.
REG-03	Entrada y salida de la misma unidad.	Estancia abierta y luego cerrada con su duración.	Pendiente	—
AVS-01	Aviso por Telegram desde la red del patio.	El aviso llega al grupo.	No	La red del patio bloquea Telegram [5].
AVS-02	Aviso por WhatsApp al destinatario principal.	El mensaje llega y se puede leer.	No	Mensajes ilegibles el 10 nov [3].
AVS-03	Aviso por WhatsApp a otro número.	El mensaje llega y se puede leer.	Sí	Prueba de aislamiento del incidente.
REN-01	Detección en modo de 40 W.	Más de 20 cuadros por segundo.	Sí	22.1 cuadros por segundo [4].
REN-02	Operación continua en el patio.	Sin fallas durante varios días.	Sí	3 días; carga y temperatura normales [5].
REN-03	Consumo en modo de 25 W.	Bajo consumo.	Sí	11 W sin unidad; 14 a 19 W con unidad [6].

Módulos pendientes de prueba (ADQ, DLN, WEB, GRB)

Tabla 6.19: Cobertura de pruebas de los módulos ADQ, DLN, WEB y GRB.

ID	Caso	Resultado esperado	Aprobado	Observaciones
ADQ-01	Se desconecta la cámara.	El sistema reintenta cada 2 s y se recupera solo.	Pendiente	Implementado en el código.
ADQ-02	Fuente de video de prueba.	Funciona igual que la cámara.	Pendiente	—
DLN-01	Lectura con vista lateral.	Número confirmado correcto.	Pendiente	Falta medir la exactitud.
DLN-02	Lectura con vista delantera o trasera.	Número confirmado correcto.	No	Según pruebas del equipo, M2 casi solo encuentra la zona con vista lateral.
WEB-01	Búsqueda y paginación de cruces.	Resultados filtrados, 10 por página.	Pendiente	—
WEB-02	Video en vivo.	10 cuadros por segundo, 960 px de ancho.	Pendiente	—

ID	Caso	Resultado esperado	Aprobado	Observaciones
GRB-01	Grabación de 72 h.	Un archivo por hora; se borran los de más de 72 h.	Pendiente	—
GRB-02	Falla la detección.	La grabación continúa.	Pendiente	—

6.1.9. Conclusiones del Prototipo 1

El Prototipo 1 cumple la cadena completa de detección, lectura, cruce, registro, avisos, tablero y grabación, y opera en vivo en el patio. La lógica de cruce es el módulo con más evidencia: pasa todos los casos del simulador, incluidos los de varias unidades a la vez que fallaban en las versiones previas. Los puntos débiles son la **entrega de avisos**, que hoy falla por la red del patio y por el incidente de WhatsApp, y la **lectura en vistas no laterales**. Además, falta evidencia formal de los módulos de adquisición, tablero y grabación, y medir la exactitud de la lectura con una muestra revisada contra el video.

Capítulo 7

Trabajo a futuro

Los resultados del Prototipo 1 definen el trabajo de los siguientes prototipos. Se agrupa por prototipo según el plan del capítulo 4.

7.1. Prototipo 2: operación continua

- **Instalación en el gabinete** del patio con la Jetson de 16 GB y fuente propia [6].
- **Arranque automático**: hoy los servicios tienen `restart: "no"` y el sistema no arranca solo al encender la Jetson [6]. Debe probarse un reinicio controlado con alguien en sitio.
- **Entrega de avisos verificada**: dar seguimiento al desbloqueo de Telegram en la red del patio [5], confirmar la solución del incidente de WhatsApp [3] y avisar cuando un canal deje de entregar mensajes.
- **Monitoreo**: avisar cuando la cámara o un servicio dejen de funcionar.

7.2. Prototipo 3: mejora de la lectura

- **Medir la exactitud** de cada etapa (detección, zona y dígitos) con una muestra de cruces revisada contra la grabación.
- **Ampliar los datos**: etiquetar los recortes guardados en el patio y versionar el conjunto de datos.
- **Reentrenar M2** con más ejemplos de vistas delantera y trasera, donde hoy casi no encuentra la zona del número.
- **Exportar** los nuevos modelos a TensorRT en la propia Jetson y comparar contra la versión anterior.

7.3. Pendientes del proyecto

- **Una sola fuente de verdad del código**: unificar la versión del repositorio, el paquete `SARU_V4` y la copia de la Jetson, y trabajar con ramas y revisión de cambios.
- **Validar con video real** la línea virtual de varios puntos (LVC-13).
- **Agregar el simulador al repositorio**, para que las pruebas LVC puedan repetirse en cada cambio.

- **Corregir la configuración**: rutas de datos por defecto que no coinciden con la Jetson, variables de Telegram exigidas aunque los avisos estén apagados y documentación del repositorio que cita scripts inexistentes [6].
- **Validar las historias de usuario** con el personal del patio y cuantificar la problemática.

Capítulo 8

Conclusiones

Conclusiones preliminares

Estas conclusiones corresponden al Prototipo 1 y se actualizarán al cerrar cada prototipo.

El Prototipo 1 de SARU demuestra que es viable registrar de forma automática las entradas y salidas de unidades en la puerta del Patio CCA con procesamiento en el borde. Una sola Jetson ejecuta tres modelos en cascada, decide los cruces, registra los movimientos, publica un tablero y graba video continuo con un consumo de entre 11 y 19 W.

La metodología de prototipado evolutivo resultó adecuada: varios de los casos más importantes, como las unidades detenidas sobre la línea, varias unidades a la vez o los números leídos con dígitos de menos, solo aparecieron al probar con video real, y se corrigieron sin detener la operación.

Los principales retos para los siguientes prototipos son tres: garantizar la entrega de los avisos desde la red del patio, mejorar la lectura en vistas no laterales y medir formalmente la exactitud del sistema. A ellos se suma un reto del proyecto: unificar las versiones del código y mantener esta documentación al día con cada cambio.

Referencias

[1] Mike Cohn. *User Stories Applied: For Agile Software Development.* Addison-Wesley, 2004.

[2] Equipo de documentación SARU. Actividad 1: reporte de entendimiento acerca del documento del proyecto, 2026. Documento interno; resume las nueve fases del proyecto.

[3] Equipo SARU. Reporte de incidente: avisos de WhatsApp sin poder leerse, 2026. Documento interno, 10 de noviembre de 2026.

[4] Equipo SARU. Reporte del código: fusión, correcciones y GitHub, 2026. Documento interno, 3 de noviembre de 2026.

[5] Equipo SARU. Reporte: sistema en vivo en el patio, 2026. Documento interno, 5 al 9 de noviembre de 2026.

[6] Equipo SARU. Reporte técnico SARU-CISA: estado del proyecto, arquitectura y componentes, 2026. Documento interno, 12 de noviembre de 2026.

[7] Google. Gemini API documentation. https://ai.google.dev/gemini-api/docs, 2026. Consultado en noviembre de 2026.

[8] Rayson Laroca, Evair Severo, Luiz A. Zanlorensi, Luiz S. Oliveira, Gabriel Resende Gonçalves, William Robson Schwartz, and David Menotti. A robust real-time automatic license plate recognition based on the YOLO detector. In *International Joint Conference on Neural Networks (IJCNN)*, 2018.

[9] NVIDIA. NVIDIA Jetson Orin. https://developer.nvidia.com/embedded/jetson-orin, 2026. Consultado en noviembre de 2026.

[10] NVIDIA. NVIDIA TensorRT developer guide. https://docs.nvidia.com/deeplearning/tensorrt/, 2026. Consultado en noviembre de 2026.

[11] OpenCV Team. OpenCV documentation. https://docs.opencv.org, 2026. Consultado en noviembre de 2026.

[12] Pallets Projects. Flask documentation. https://flask.palletsprojects.com, 2026. Consultado en noviembre de 2026.

[13] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[14] Jithmi Shashirangana, Heshan Padmasiri, Dulani Meedeniya, and Charith Perera. Automated license plate recognition: A survey on methods and techniques. *IEEE Access*, 9:11203–11225, 2021.

[15] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

[16] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2016.

[17] SQLite Consortium. SQLite documentation. https://www.sqlite.org/docs.html, 2026. Consultado en noviembre de 2026.

[18] Telegram. Telegram bot API. https://core.telegram.org/bots/api, 2026. Consultado en noviembre de 2026.

[19] Ultralytics. Ultralytics YOLO documentation. https://docs.ultralytics.com, 2026. Consultado en noviembre de 2026.

[20] WhiskeySockets. Baileys: WhatsApp web API for JavaScript/TypeScript. https://github.com/WhiskeySockets/Baileys, 2026. Consultado en noviembre de 2026.

[21] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, pages 1–21, 2022.

[22] Karel Zuiderveld. Contrast limited adaptive histogram equalization. In *Graphics Gems IV*, pages 474–485. Academic Press, 1994.

Apéndice A

Evidencias de pruebas del Prototipo 1

Este apéndice reúne la evidencia de cada caso del protocolo de pruebas (sección 6.1.8). Cada evidencia se identifica con el ID del caso: una captura de pantalla, un fragmento del registro del sistema o una foto del aviso recibido.

Cómo agregar una evidencia

Guardar la imagen como `figuras/A_evidencias_p1/<ID>.png` (por ejemplo, `LVC-01.png`) e insertarla en la subsección de su módulo con `\figura`. Las evidencias no deben mostrar tokens, números de teléfono reales ni personas identificables.

A.1. LVC: línea virtual y cruce

Casos LVC-01 a LVC-12: resultados del simulador reportados en [4] y [6]. Falta anexar la salida del simulador.

A.2. REG, AVS y REN: registro, avisos y rendimiento

Casos con resultado registrado en los reportes del equipo [4, 5, 3, 6]. Falta anexar las capturas correspondientes.

A.3. ADQ, DLN, WEB y GRB

Sin evidencia todavía: los casos están pendientes de ejecutar.

Apéndice B

Glosario

En los reportes y en el código, un mismo concepto aparece con nombres distintos; por ejemplo, el "panel de detecciones" del tablero lista cruces registrados. Este glosario fija **un término por concepto**. Si un documento, una pantalla o el código usan otra palabra para lo mismo, se corrige el otro, no el glosario. Debajo de cada término, *Evitar* lista las palabras que causan confusión.

B.1. Proyecto

SARU
: Sistema Automatizado de Reconocimiento de Unidades: el proyecto y el sistema que registra las unidades en la puerta del Patio CCA. **Por definir** si el nombre oficial es SARU o SARU-CISA.
Evitar: CISA, VisionCISA, "el sistema de visión".

Grupo CISA
: Organización responsable del proyecto.
Evitar: CISA a secas para referirse al sistema.

Patio CCA
: Patio del Metrobús donde opera SARU.

Puerta
: Acceso del patio vigilado por una cámara.

B.2. Unidades

Unidad
: Autobús del Metrobús que entra o sale del patio.
Evitar: camión, bus, autobús (en texto técnico).

Número económico
: Número pintado en la unidad que la identifica.
Evitar: placa, ID, número de unidad.

Padrón
: Lista oficial de unidades válidas, con su empresa y su tipo.
Evitar: catálogo, base de unidades.

Fuera del padrón
: Estado de un número económico leído de forma consistente que no aparece en el padrón.
Evitar: no registrado, desconocido, error de lectura.

Empresa operadora
Empresa a la que pertenece una unidad.

B.3. Movimiento

Línea virtual
Trazo sobre la imagen de la cámara que separa el exterior del interior del patio.
Evitar: umbral, línea de conteo.

Cruce
Paso confirmado de una unidad a través de la línea virtual, con dirección de **entrada** o de **salida**.
Evitar: detección, evento, paso, movimiento.

Maniobra
Acercamiento de una unidad a la línea virtual seguido de un regreso, sin completar el cruce.
Evitar: falso cruce.

Estancia
Tiempo que una unidad pasa dentro del patio, desde su entrada hasta su salida.
Evitar: visita, permanencia.

Día operativo
Jornada del patio que empieza a las 3:00 a.m. y termina a las 2:59 a.m. del día siguiente.
Evitar: día (a secas) al hablar de conteos.

B.4. Lectura

Detección
Hallazgo de una unidad en un cuadro de video.
Evitar: usarla para referirse a un cruce registrado (el "panel de detecciones" del tablero lista cruces).

Zona
Parte de la unidad donde está pintado el número económico: delantera, lateral o trasera.
Evitar: vista, región.

Lectura
Número que el sistema obtiene de una zona en un solo cuadro.

Número confirmado
Número económico respaldado por varias lecturas iguales de la misma unidad.
Evitar: número detectado.

M1, M2, M3
Los tres modelos de visión, en cadena: M1 detecta unidades, M2 localiza la zona y M3 lee los dígitos.

Recorte
Imagen de una unidad o de una zona, extraída de un cuadro para revisión o entrenamiento.
Evitar: crop (en texto en español).

B.5. Avisos y evidencia

Aviso
: Mensaje con fotos y datos de un cruce, enviado por Telegram o WhatsApp.
Evitar: alerta, notificación.

Validación
: Revisión de un cruce, por Gemini o por un validador, antes de darlo por bueno.

Validador
: Persona autorizada para confirmar o corregir una lectura dudosa.

Grabación
: Video continuo de la cámara de la puerta, guardado en archivos de una hora durante 72 horas.